

8,1
8,1 (P2)

UNIVERSIDADE FEDERAL DE CATALÃO
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO
CURSO DE CIÊNCIA DA COMPUTAÇÃO

- Aluno: _____
- Data: 13/09/2024
- Prof.: Ricardo Couto A. da Rocha

2ª Prova de Linguagens de Programação

1. Ao final da prova, você terá que entregar todas as folhas, incluindo o enunciado.
2. Coloque nome em todas as folhas, incluindo na folha de enunciado.
3. As questões podem ser feitas em qualquer ordem e podem ser feitas a lápis (**exceto** questões objetivas)
4. **Todas** as questões devem ser feitas **nas folhas em anexo**, mesmo as respostas das questões objetivas.

Boa prova!

1ª Questão (2,0 pontos): Suporte a Orientação a Objetos

Considere o código C++ da figura a seguir, que declara as classes **A**, **B**, **C** e **D**, e cria diversos objetos instâncias dessas classes. Observe que os trechos “: **A(a)**” (linha 22 e 52) e “: **B(a, b)**” (linha 38), indicam que os respectivos construtores das superclasses (classes pai), estão sendo invocados.

```
1  class A {
2      private:
3          int _a;
4      public:
5          A(int a) {
6              this->_a = a; }
7          int m1() {
8              return 1;
9          }
10         virtual int m2() {
11             return 2;
12         }
13         int getA() {
14             return _a;
15         }
16     };
17
18     class B : public A {
19     private:
20         int _b;
21     public:
22         B(int a, int b) : A(a) {
23             _b = b;
24         }
25         int m1() {
26             return 3;
27         }
28         int m3() {
29             return 4;
30         }
31         int getB() { return _b; }
32     };
33
34     class C : public B {
35     private:
36         int _c;
37     public:
38         C(int a, int b, int c) : B(a, b) {
39             this->_c = c;
40         }
41         int m2() {
42             return 5;
43         }
44         int getC() { return _c; }
45     };
46
47     class D : private A {
48     private:
49         int _d;
50     public:
51         D(int a, int d) : A(a) {
52             this->_d = d;
53         }
54         int m2() {
55             return 6;
56         }
57         int getD() { return _d; }
58     };
59
60     int main() {
61
62         A a(1), *ptr_a;
63         B b(2,3), *ptr_b;
64         C c(4,5,6), *ptr_c;
65         D d(7,8), *ptr_d;
66
67         ptr_a = new A(1);
68         ptr_b = new B(2,3);
69         ptr_c = new C(4,5,6);
70         ptr_d = new D(7,8);
71     }
```

1.1. Considerando os códigos e inicializações da figura abaixo, responda para as questões a seguir, de (a) a (j), se a instrução C++ é VÁLIDA ou INVÁLIDA.

Caso seja VÁLIDA, indique o valor avaliado ou conteúdo do objeto. Caso seja INVÁLIDA (causando possível erro de compilação), justifique a sua resposta. A sua resposta deve NECESSARIAMENTE seguir o padrão: "VÁLIDO: <valor retornado da respectiva avaliação>" ou "INVÁLIDO: <explicação de porque a linha é inválida>".

```

1 A a2 = c; // a)
2 c = a; // b)
3 ptr_c = &a; // c)
4 A *ptr_a2 = new C(4,5,6);
5 c.m1(); // d)
6 b.m2(); // e)
7 c.m2(); // f)
8 a.m1();
9 b.m1();

```

```

10 d.m1();
11 d.m2();
12 ptr_a->m1();
13 ptr_b->m1(); // g)
14 ptr_d->m1(); // h)
15 ptr_d->m2(); // i)
16 a = b;
17 a.getB();
18 A *ptr_a3 = new D(7,8); // j)

```

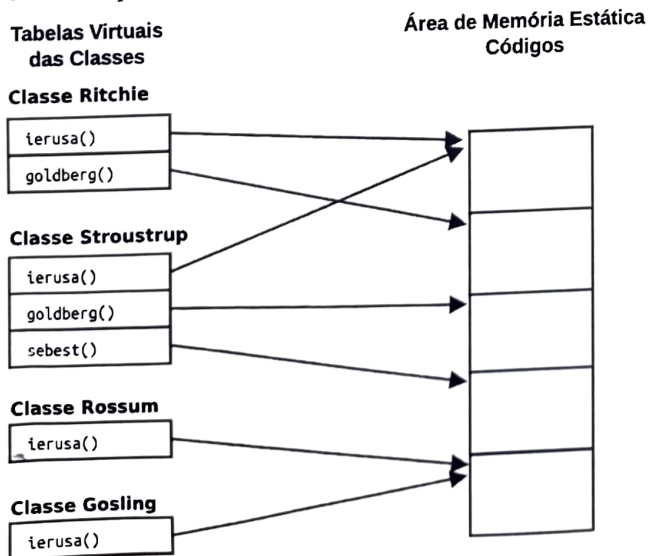
- A declaração e inicialização da linha 1
- A atribuição da linha 2
- A atribuição da linha 3
- A invocação da linha 6
- A invocação da linha 7
- A invocação da linha 8
- A invocação da linha 14
- A invocação da linha 15
- A invocação da linha 16
- A declaração e inicialização da linha 19

1.2. Para as invocações inválidas indicadas nas respostas anteriores, indique quais invocações seriam válidas em um código equivalente das linguagens abaixo, justificando.

- Java
- Python

2ª Questão (2,0 pontos): Implementação de Abstrações de Orientação a Objetos

Considere que o código da figura abaixo, semântica e sintaticamente correto, em que objetos ptr_Stroustrup, ptr_Gosling, ptr_Rossum e ptr_Ritchie são ponteiros para objetos das classes Stroustrup, Gosling, Rossum e Ritchie, respectivamente. Considere que essas são as únicas classes do cenário e que NÃO HÁ herança múltipla.



```

1 C1 *ptr_Rossum = new Rossum();
2 C2 *ptr_Ritchie = new Ritchie();
3 C3 *ptr_Stroustrup = new Stroustrup();
4 C4 *ptr_Gosling = new Gosling();
5
6 ptr_Rossum = ptr_Gosling; // atribuicao 1
7 ptr_Ritchie = ptr_Stroustrup; // atribuicao 2
8 ptr_Rossum = ptr_Stroustrup; // atribuicao 3
9 ptr_Rossum->cardelli();

```

Escreva um código mínimo C++ para as classes Stroustrup, Gosling, Rossum e Ritchie, indicando as relações de herança, métodos virtuais e não-virtuais. Os métodos não devem ser implementados e não é necessário indicar os construtores.

3ª Questão (2,0 pontos): Subprogramas

Considere a seguinte implementação recursiva de uma função para calcular o *n*ésimo número de Fibonacci.

```

1 int fib(int n) {
2     if(n == 0) {
3         return 0;
4     }
5     else {
6         if(n == 1) {
7             return 1;
8         }
9         else {
10            return fib(n - 1) + fib(n - 2);
11        }
12    }
13 }

```

2
1
1 0

- a) Descreva as operações na pilha de execução do programa para a chamada `fib(3)`, em termos de chamadas `empilha(fib(k))` e `desempilha(fib(k))`, onde *k* é o valor do parâmetro passado para a respectiva chamada. Coloque uma chamada por linha da sua resposta.
- b) Considerando que um inteiro ocupa 4 bytes nessa linguagem, e considerando **APENAS** o espaço ocupado pelo parâmetro e variáveis locais na AR da função, qual é o maior número de bytes que a execução de `fib(3)` e `fib(5)` ocupam na memória? Coloque as respostas para cada uma das chamadas.

4ª Questão (2,0 pontos): Avaliação em Curto-Circuito

Considere um programa escrito em uma linguagem de programação qualquer que está com um bug no trecho de código da figura. Você suspeita que esse problema está ocorrendo porque a linguagem em questão não possui avaliação em curto-circuito.

```

if (arquivo and arquivo.pronto()) {
    # faca algo
}

```

- a) Escreva um trecho de pseudo-código para testar se a linguagem oferece avaliação em curto-circuito. Por exemplo, se ela oferece curto-circuito o código escreve “*X oferece curto-circuito!*” do contrário ela escreve “*Melhor programar em outra linguagem!*”.
- b) Escreva um trecho de pseudo-código em substituição à comparação com problema, que funcione mesmo que não haja avaliação curto-circuito na linguagem.
- c) Como esse tipo de comparação ocorre frequentemente no seu código, um outro programador sugeriu que você implementasse uma função `ANDcurtocircuito(...)` que incluísse o código feito no item anterior e você pudesse replicar em todo o seu programa. Se essa solução funcionar, descreva um trecho de pseudo-código (considerando apenas dois parâmetros). Caso a solução não funcione, explique a razão.

5ª Questão (2,0 pontos): Assinale a alternativa CORRETA em cada uma das questões objetivas abaixo.

5.1. Considere os seguintes código abaixo:

```

1 int test1(int* v) {
2     return v[0] == 0; ✓
3 }
4
5 float circunf(int raio) {
6     return 2 * pi * raio; ✓
7 }
8
9 void libera() {
10     n_abertos--; ✓
11 }
12
13 int abre(F* f) {
14     if (f != NULL) {
15         f->aberto = 1;
16         return 1;
17     }
18     else {
19         return 0;
20     }
21 }

```

Quais dessas funções produzem efeito colateral?

- a) libera()
- b) abre() e libera()
- c) test1() e abre()
- d) test1(), abre() e libera()
- e) Em todas as funções.
- ☒ f) Em nenhuma das funções.

5.2. Em relação à amarração entre parâmetros formais e reais

- a) Parâmetros variáticos não podem ser implementados por meio de amarração posicional. ✓
- b) A amarração posicional de parâmetros pode ser comportar como amarração nomeada mas o contrário não é possível. ✓
- c) Na amarração nomeada, cada parâmetro real deve, além de respeitar a sua ordem no parâmetros formais, estar associado aos nome do parâmetro formal. ✗
- ☒ d) A implementação de amarração posicional de parâmetros é mais eficiente que a amarração nomeada.
- e) Quando a quantidade de parâmetros em um subprograma é grande, a amarração posicional se mostra mais adequada que a nomeada, tanto quanto à legibilidade quanto à redigibilidade. ✗
- f) Quando a quantidade de parâmetros em um subprograma é grande, parâmetros variáticos são mais adequados que a amarração posicional, especialmente quanto à legibilidade. ✗

5.3. Um subprograma ordena(lista, comparador) espera como segundo parâmetro um outro subprograma. Em um código que invoca ordena

- a) ele deve informar os parâmetros que serão passados a comparador, na forma de outros parâmetros.
- b) ele deve realizar a invocação de comparador com seus respectivos parâmetros.
- ☒ c) o parâmetro comparador deve ser unicamente o nome do subprograma ao qual está associado.
- d) o parâmetro real associado a comparador não deve ser especificado. O parâmetro é especificado na própria especificação do subprograma ordena que o invocará. ✗

5.4. Quando ocorre uma coerção de tipo em linguagens de programação?

- ☒ a) Quanto o conteúdo de uma variável precisa ser convertida para outro tipo para poder ser avaliado em uma expressão.
- b) Quanto o conteúdo de uma variável precisa ser convertida para outro tipo para que o trecho de código seja compilável ou interpretável.
- c) Quanto o código informa explicitamente, segundo indicado pelo programador, um tipo para qual o conteúdo de uma variável precisa ser modificado. ✓
- d) Quando uma expressão está sendo avaliada em uma variável de tipo incompatível com um operador.

- a) Valido: C é subclasse de B que por sua vez é subclasse de A, o que permite a atribuição de "c" para "a" que é do tipo A.
- b) Invalido: "c" é do tipo C que é subclasse de A, sendo a do tipo A não é possível ~~c = a~~ devido a logica das linguagens orientadas a Objetos.
- c) Invalido: ptr-c é um ponteiro do tipo C que é subclasse de A, logo não pode apontar para "a" ~~contendo~~ que é do tipo A.

1. 9 d) Valido: 3
e) Valido: 2

f) Valido: 5
g) Valido: 3

h) Invalido: ~~A invocação~~ A classe D estende a classe A de maneira privada e não permite chamar o método m1.

i) Valido: 6

j) Valido: O ptr-a3 aponta para um objeto do tipo D, só que ptr-a é um ponteiro para A e D é subclasse de A.

1. 2- a) Em java todas as atribuições Invalidas devido a lógica de programação de objetos (b, c) continuariam invalidas. Porém a invocação "h" seria valida uma vez que não há como estender uma classe de forma privada em Java.

b) Em python as invocações (b, c) seriam validas pois como a anotação de tipos é dinamica o tipo da variavel se transforma assim que é realizada a atribuição. ~~Essa transformação de tipo também afeta a invocação h, pois~~ No caso da atribuição h ela também se torna valida pois não tem o limite imposto pelo private.

```
2- class Rossum {
    Public:
        Cardelli();
        Virtual cerusa();
}
class Gosling: public Rossum {
    Public:
        Virtual cerusa();
        Virtual goldberg();
}
class Ritchie: public Rossum {
    Public:
        Virtual cerusa();
        Virtual goldberg();
}
```

```
class stroustrup() : public Ritchie {
    Public:
        Virtual cerusa();
        Virtual goldberg();
        Virtual sebest();
}
```

3- a) empilha (fib(3))
 empilha (fib(2))
 empilha (fib(1))
 desempilha (fib(1))
 empilha (fib(0))
 desempilha (fib(0))
 desempilha (fib(2))
 empilha (fib(1))
 desempilha (fib(1))
 desempilha (fib(3))

20

b) fib(3) = 12 bytes
 fib(5) = 20 bytes

4) a) Saída = "Linguagem oferece curto circuito!"
 bool Ruim {
 Saída = "Melhor programar em outra linguagem!"
 Return true;

0,4

if (False and Ruim) {

Faça algo }

Print(F { Saída })

vai imprimir de qualquer maneira

1,4
 2

b) if (arquivo) {

if (arquivo.pronto()) {

faça algo

0,7

c) bool ANDcircuito(x, y) {

if (x) {

if (y) {

Return true;

else {

Return false;

else {
 Return false;

5- 5.1 e + X
 5.2 d D ✓
 5.3 c C ✓
 5.4 d D X

10